An arithmetic ceiling on issue–commit linkage, and what it means for corpus selection

Anonymous Author(s)

Abstract

Empirical studies of architectural change read a project's own records and assume the record traces the work. We measure that assumption on 38 Apache projects.

Two traceability rates diverge. The commit-side rate, the one the literature reports, asks whether a commit cites a ticket. The ticket-side rate asks whether a ticket ever receives a commit, which is what a ticket-anchored design actually needs. Hive cites a ticket in **97.0%** of its commits while only **55.8%** of its tickets are ever cited by one.

The divergence is arithmetic rather than a matter of discipline. A project supplying c commits per ticket cannot realise more than a c fraction of its tickets whatever its commit-side rate. Across the twelve eligible projects this ceiling varies **6.0-fold** while the share of it actually reached varies **1.19-fold**, so nearly all the spread is supply rather than behaviour. No published linkage rate reports commits per ticket beside it.

Against a bar of 0.80 pre-registered before any project was cloned, **12 of 38** qualified and none outside a single ecosystem cleared it. We give six mechanisms by which a project silently fails to support an issue-linked study, and none of them is expressible in any published sampling frame.

CCS Concepts

• **Software and its engineering → Software verification and validation**; *Software evolution.*

Keywords

traceability, corpus eligibility, issue linkage, sampling frames, mining software repositories

1 Introduction

Empirical work on architectural change reads a project's own records — commit messages, issue trackers, pull requests — and assumes the record is a reasonably faithful trace of the work. Studies of refactoring motivation, of technical-debt repayment, of architectural erosion and of review effort are all built on that assumption, and it is rarely stated, still more rarely measured. This paper measures it on 38 Apache projects, and finds that the record traces the work far less often than a ticket-anchored design needs, for a reason that is arithmetic rather than cultural.

How we arrived at a methods paper. This study set out to measure signal-to-action latency: what in a project's records precedes a decision to restructure code. Six operationalisations were tried and closed, and Figure 1 records them. The sampling frame turned out to be the finding, and this paper reports that rather than the latency result it went looking for.

Contributions. Stated against what is already published rather than against an assumed gap. We make **no first-to-measure claim**: Rath & Mäder's SEOSS 33 [8] publishes per-project linkage as an explicit table column for 33 projects, and Rath et al. (ICSE'18) [9] publishes both directions for six. An earlier version of this work claimed novelty on that ground; the claim was checked, found false, and withdrawn (§ 2.1).

(1) **A pre-registered numeric eligibility bar, applied before any outcome, with reported attrition and the rejected candidates named.** Prior work selects on trace links informally — "a project shall continuously capture trace links", "each largely followed the practice of tagging commits with issue IDs" — and reports rates afterwards. Neither states a threshold, an attrition count, or which candidates were rejected. Ours is fixed at 0.80, committed before any project was cloned, and never moved.

(2) **A population finding: 12 of 38 pass, and no project outside one ecosystem clears the bar.** The best outside it, James, reaches 74.8%, against a median of 63.6% across all 38 and 44.8% across the 26 rejected. The ecosystem label is a hand classification and we say so: two measured generational variables were tested in its place and neither separates the corpus as well (§ 4.1). SEOSS's own table contains the ingredients — Apache projects at the top, JBoss at the bottom — but the inference is not drawn there.

(3) **Both reference channels measured together.** Counting GitHub-issue references alongside Jira keys is what shows that the bar selects a *tracker* rather than a discipline: seven of the dropped projects cite GitHub issues in more than half of their commits.

(4) **The arithmetic ceiling that bounds the ticket-side rate**, and the decomposition of that rate into a ceiling set by commits-per-ticket and a residual that measures citation discipline. In this corpus the residual is nearly constant, which is what the divergence actually consists of.

(5) **The ticket-side rate measured across the whole probe rather than only the survivors** — including the 26 projects that failed the commit-side bar. A strong positive association between the two rates is ruled out; a null is not established, and the live and frozen measurements disagree in sign (`paper/DIRECTION_TENSION.md`).

(6) **A six-mode taxonomy of the ways a project silently fails to support an issue-linked study**, three of them disqualifying and three of them producing a plausible wrong number rather than an error.

2 Related work

2.1 Per-project linkage rates are already published

This paper makes no first-to-measure claim, and the claim it originally made was withdrawn. Two prior works publish per-project issue–commit linkage rates, one of them as an explicit table column.

Rath & Mäder 2019, SEOSS 33 [8] (*Data in Brief* 25:104005) publishes, per project, change-set count and **"Linked Change Sets [%]"** for 33 projects — the same quantity as our commit-side rate. Their spread is 8.11%–97.13%; ours is 0.01%–98.3% across 38. Four of five overlapping projects agree within **3.3pp** on corpora seven years apart:

That agreement is **independent cross-corpus validation of the measure** and is treated here as such. Flink's 24.1pp gap is scope rather than error, and unlike in earlier drafts this is now tested rather than asserted: restricting our probe to the **earliest 12,419 commits** — the change-set count SEOSS publishes, and the only quantity the two studies share — gives **5,214 / 12,419 = 41.9841%** against their **41.98%**, a gap of **+0.0041pp**. All five overlapping projects agree once scope is matched.

SEOSS's selection criteria matter for our argument: a project must "continuously capture vertical and horizontal trace links among these artifacts". So a traceability criterion **is** used for selection — but as a qualitative requirement. No cut-off is stated, no rejected candidates are reported, and having selected on trace links the dataset still admits Errai at 8.11%.

Rath et al., ICSE 2018 [9] (*Traceability in the Wild*, arXiv:1804.02433) publishes **both directions** for six Git+Jira projects, and is the closest precedent to our divergence result. Commit side: "approximately 48% of the commits were not linked to any issue", with a per-project spread from 15% unlinked in Derby to ~76% in Maven. Ticket side: "approximately 43.3% of improvements and 42.4% of bugs have no commits associated with them" — Derby works out to 51.8% ticket-side coverage. Our 12 passing projects run 12.0%–69.0% ticket-side, straddling their figure. Their sentence — "different practices exist across different projects, leading to huge disparities in the extent to which issue tags are added to commit messages" — is our corpus-feasibility finding stated qualitatively in 2018.

Selection there was also informal: the six were chosen because each "largely followed the practice of tagging commits with issue IDs". Again no threshold.

2.2 The standard sampling frame cannot express the criterion

Dabic et al. 2021, GHS [4] (*Sampling Projects in GitHub for MSR Studies*, MSR'21) is the standard sampling tool and indexes **735,669 repositories**. A record carries **35 fields**, queried from the live API rather than read off the paper's Table I, which is an image. Only **`totalIssues`** and **`openIssues`** touch issues at all, and both are GitHub-issue counts. There is no field for issue-tracker type, external tracker usage, issue–commit linkage, traceability, or commit-message convention. The one adjacent feature, filtering by issue label, is described in the paper as "still under development".

The consequence is the actionable one: a researcher sampling with GHS gets stars, commits, contributors and license, then discovers post hoc that 26 of 38 candidates are unusable. That is what happened here.

2.3 The bug-side linkage literature, and what it already settled

The commit-side/ticket-side asymmetry measured here was studied on the bug side a decade and a half ago. **Bachmann et al. (FSE'10)** [1] had a core Apache HTTP Server developer annotate **493 commits** exhaustively with a purpose-built tool [3], establishing ground truth rather than inferring it, and found only **47.6%** of bug-fix-related commits documented in the tracker; their target is the completeness of the link on one project over one window, ours a per-project rate over a whole tracker, established mechanically. **Bird et al. (ESEC/FSE'09)** [2] is why it matters: missing links are not missing at random, so a dataset built from linked records is a biased sample and models fitted to it inherit the bias, which is the same argument this paper makes for architectural change. Around them, Herzig et al. (ICSE'13) [5] show the reports themselves are misclassified, Nguyen et al. (WCRE'10) [7] replicate the bias result, and a line of work beginning with ReLink [11] recovers missing links from time proximity, author identity and textual similarity. The two measurements do not condition on the same thing: Bachmann et al. restrict attention to bugs and bug-fix commits, whereas § 3.1.1 admits every issue type and conditions on nothing. This paper does not adjudicate whether that makes the quantities distinct, and no claim it makes depends on the answer.

2.4 Refactoring and self-admitted technical debt

The successor design this corpus limit points towards is a self-admitted technical debt anchor, and its novelty margin rests on three simultaneous choices that the prior work does not combine. That design is not proposed here.

3 Method

3.1 Two traceability rates, and only one of them is published

The literature reports one number and issue-anchored studies depend on another. Both are stated here as definitions so that the divergence in § 4 is a comparison between named quantities rather than between two informal readings of the word "traceability".

Let p be a project with a set of Jira project keys K_p, a git repository observed at a pinned commit H_p, and a tracker read at a snapshot time **T**. Write `cites(c)` for the set of issue keys appearing in commit c's message.

Definition 1 — commit-side rate. The fraction of commits reachable from H_p whose message cites at least one key with a prefix in K_p:

$$\mathrm{CSR}(p) = \frac{\left|\{c : c \longrightarrow H_p, \mathrm{cites}(c) \cap K_p \neq \emptyset\}\right|}{\left|\{c : c \longrightarrow H_p\}\right|}$$

This is the quantity Rath & Mäder publish per project as *"Linked Change Sets [%]"* (SEOSS 33 [8]) and the quantity Rath et al. (ICSE'18) [9] report as "approximately 48% of the commits were not linked to any issue". It is what the corpus-selection bar in this study is defined on, and it is not novel here.

Definition 2 — ticket realisation rate (TRR). The fraction of the project's tracked issues that are ever cited by at least one commit:

project	SEOSS 2019	this probe 2026	Δ
Hadoop	97.13% (27,776 commits)	97.8% (28,290, 7-key)	+0.7pp
Hive	96.34% (11,179)	97.0% (18,213)	+0.7pp
HBase	90.06% (14,331)	92.5% (21,220)	+2.5pp
ZooKeeper	87.12% (1,600)	90.4% (2,718)	+3.3pp
Flink	41.98% (12,419)	66.0% (38,219)	**+24.1pp**

$$\mathrm{TRR}(p) = \frac{\left|\{k \in \mathrm{Tickets}(p,T) : \exists c \longrightarrow H_p, k \in \mathrm{cites}(c)\}\right|}{|\mathrm{Tickets}(p,T)|}$$

TRR is a label of convenience for this paper. It is not a claim to have named the quantity first, and nothing here depends on the name being new. The quantity has been measured before: Rath et al. (ICSE'18) [9] report that "approximately 43.3% of improvements and 42.4% of bugs have no commits associated with them", as a property of an issue type rather than as a project-level rate, and Bachmann et al. (FSE'10) [1] measure the completeness of the bug-side link against expert ground truth. What is new here is the use made of the quantity rather than the quantity itself: the divergence between TRR and CSR is not framed anywhere as a constraint on corpus selection.

"Realisation" is a claim about the record, not about the work. TRR counts a ticket as realised when the repository's own record points back to it. A ticket can be resolved, implemented and shipped without any commit naming it, and such a ticket is counted as unrealised. That is the intended reading: the measure is of what a researcher can *recover*, not of what a project *did*.

3.1.1 What counts as a ticket. Denominator membership is deliberately permissive, because a ticket-anchored study samples from the whole tracker before it knows which tickets are useful:

- **All issue types.** Bug, Improvement, New Feature, Task, Test, Wish and Sub-task alike. **Sub-tasks are included**, because they carry their own key and are cited in commit messages in their own right; excluding them would drop the keys most likely to be cited and inflate the rate.
- **All statuses and resolutions.** Nothing is conditioned on `resolution = Fixed`. This matters for comparability: Vieira et al. (PROMISE'19) [10] select on resolution before measuring, which pre-selects tickets that were worked, so any rate computed that way is not comparable to TRR without adjustment.
- **Keys, not names.** Membership is by the issue's key prefix. Measured across the 2,686,282-issue public Jira corpus, 326 project ids carry more than one distinct name while 0 carry more than one key, so a name-based denominator is not stable and a key-based one is (`paper/numbers.md` §5).
- **Issues that left the project are not in it.** An issue moved out of p before T no longer has a p-key in the tracker and is not counted; one moved in is counted under its current key. This is a property of the snapshot, not a choice.

3.1.2 What counts as "ever received a commit". A ticket is realised if **at least one** commit reachable from H_p carries a token matching `\b(?:K1|K2|...)-\d+\b` equal to its key, in the commit **subject or body**. Nothing else is read: not the diff, not the branch name, not a pull request body, not the tracker's own link fields.

The matcher's precision was measured rather than assumed: **195 of 200 = 97.5%** (Wilson 95% CI 94.3–98.9%) on a seeded 200-commit manual sample across the 12 eligible projects, corpus-weighted 97.7% (`paper/matcher_validation.md`). Two of the five enumerated failure categories could not have fired — `version_string` is near-unreachable given a pattern demanding the upper-case key followed by `-` and digits, and `foreign_key` is unreachable for 11 of the 12, because each project is probed with its own key set and only Ozone is multi-key with both keys its own. **The single-key → multi-key result on Hadoop is therefore not validated by that sample**, Hadoop being the only true monorepo in the study and not among the 12.

One construct limit is recorded rather than corrected: **a key matched from a branch name counts as a citation.** Merge subjects (`Merge branch 'master' into KNOX-998-Package_Restructuring`) and svn trailers (`.../branches/TEZ-1@1471779`) both occur in the sample. The ticket association is correct; the commit need not implement the ticket. The five failure categories were fixed before labelling and do not cover this case, so it was counted genuine and is disclosed here instead of being assigned an invented category.

3.1.3 How the denominator is bounded in time. TRR compares a tracker read at T against a repository read at H_p, and **the measure requires T ≤ date(H_p)** so that every ticket in the denominator has had the whole interval to date(H_p) in which to receive a commit. A tracker read ahead of the repository makes the newest tickets structurally incapable of being realised and biases TRR downward by an amount that depends on the project's filing rate, which looks exactly like poor traceability. Two instantiations are used and both satisfy it: $\mathrm{TRR}_{\mathrm{live}}$ reads Apache Jira and the pinned shas on the same day, 2026-07-25, restricted mechanically to issue keys so that nothing in it can be turned into a duration; $\mathrm{TRR}_{\mathrm{frozen}}$ uses the Public Jira Dataset [6], a snapshot predating every pinned sha, with membership defined in issue-number space because the snapshot's per-project date is not recoverable. Throughout, an unsubscripted `TRR`, `ceiling` or `fill` means the live measurement.

3.1.4 The arithmetic ceiling, and what is left once it is removed. CSR and TRR are not independent quantities, and the constraint between them is exact rather than statistical. It is stated here because §4.2 and §4.3 are organised around it.

A commit either cites no key of p, in which case it contributes nothing to the numerator of TRR, or cites at least one — and however many it cites, it can add **at most one ticket that no earlier commit had already cited**, in the limiting case where every citing commit introduces a fresh key. So the number of distinct realised tickets is bounded by the number of citing commits:

$$\left|\{k \in \mathrm{Tickets}(p,T) : k \text{ realised}\}\right| \le \mathrm{CSR}(p) \cdot |C_p|$$

and dividing by |Tickets(p, T)|:

$$\mathrm{TRR}(p) \le \frac{\mathrm{CSR}(p) \cdot |C_p|}{|\mathrm{Tickets}(p,T)|} \equiv \mathrm{ceiling}(p)$$

The bound is tight — equality holds when citing commits map one-to-one onto distinct previously-uncited tickets — and it is reached in practice only if no ticket ever receives two commits.

Two consequences, and the second is why this matters.

First, **`commits / tickets` is a scale factor the two rates do not share.** A project with 0.16 commits per ticket cannot exceed a 16% ticket-side rate at *any* commit-side rate, including 100%. Comparing CSR and TRR without it compares a ratio to a ratio with a different denominator.

Second, it decomposes TRR into a part that is arithmetic and a part that is behaviour:

$$\mathrm{fill}(p) = \frac{\mathrm{TRR}(p)}{\mathrm{ceiling}(p)} \in [0,1]$$

`fill` is the share of the attainable maximum actually reached — how efficiently a project's citing commits spread across distinct tickets rather than piling onto a few. **`fill` is the quantity a claim about citation discipline needs**; `ceiling` is a property of how much code the project writes per ticket it files. Both are reported for every eligible project (Table 1). § 4.2 shows that in this corpus almost all of the ticket-side variation is ceiling and almost none of it is fill.

The subscript propagates, and it matters. `ceiling` and `fill` are both functions of TRR and of the ticket denominator, so each inherits whichever instantiation of § 3.1.3 produced it: $\mathrm{ceiling}_{\mathrm{live}}$ and $\mathrm{fill}_{\mathrm{live}}$ are computed against the live Jira read of 2026-07-25, $\mathrm{ceiling}_{\mathrm{frozen}}$ and $\mathrm{fill}_{\mathrm{frozen}}$ against the frozen snapshot. The two are not interchangeable and they do not always agree about whether the bound binds at all — three projects have $\mathrm{ceiling}_{\mathrm{frozen}}$ at or above 100% while every $\mathrm{ceiling}_{\mathrm{live}}$ is below it (§ 4.2, the artifact's 38-project table). **Throughout this paper an unsubscripted `TRR`, `ceiling` or `fill` means the quantity in general rather than a measured value; every measured value carries its subscript**, in the prose, in the abstract and in the table captions. Where a project appears with two numbers for what looks like one quantity, the subscripts are the difference.

3.2 Corpus construction

38 Apache candidates, each Maven-built, Jira-tracked and multi-module. Each was cloned `--filter=blob:none --no-checkout` — the probe needs `git log` and nothing else, so a working tree is pure cost — and probed with `scripts/traceability_probe.py`. Per-project HEAD shas are pinned in `paper/traceability_probe.json`, so any re-run is exactly diffable against this one.

Hadoop itself is not one of the 38. It is the corpus the exploratory work was done on, and it is used here only as the worked example for multi-key matching. Every rate reported for the eligible corpus is from a project Hadoop is not.

Key prefixes were detected empirically from commit messages, not assumed. This is not a refinement; it decides the answer. A single-key probe reads Hadoop at **26.2%**, against **92.3%** on a four-key probe and **97.8%** on a seven-key probe of the same 28,290 commits (§ 5, modes 5 and 6). The three figures are the same measurement under three key sets, not a rate and a correction to it; 97.8% is the most complete of them, and the paper quotes whichever key set it names. Evergreen reads 71.7% under a single-key probe for the same reason. Seven of the 38 needed a second key (`CALCITE,OPTIQ`, `HDDS,OZONE`, `KARAF,FELIX`, `PINOT,THIRDEYE`, `ROCKETMQ,RIP`, `TOMEE,OPENEJB`, `JAMES,MAILBOX`).

One of those second keys was never used. `OZONE` appears in Ozone's probed key set and is cited by **zero** commits; every Jira reference in that repository is an `HDDS` key. It is retained in the probe because the key set was fixed from a prefix scan before the counts were read, and removing it afterwards would be selection on the outcome. It is also one of **six** probed keys with no project record in the frozen tracker corpus, which is a different fact about the same key: it is neither cited in git nor present in the tracker.

Both reference channels are counted. GitHub-issue references (`#NNN`, `GH-NNN`) are counted per repository alongside Jira keys. This is what turns each drop reason from an inference into a measurement: `github_issue_references_dominate` means the GitHub count exceeds the Jira count in that repository, and nine projects qualify.

3.3 The bar, and the discipline around it

CSR ≥ 0.80, fixed in `predictions/PREDICTIONS.md` (`ca076a9`) **before any project was cloned, and never moved.** Lowering it to 60% would have admitted nine more projects and widened the corpus beyond a single ecosystem. It was not lowered.

The same discipline was applied where it hurt more. A separate frozen threshold — vendor share ≥ 0.40 for the module-tier rule — was held through a held-out replication in which lowering it would have rescued two of five projects (HBase, maximum share 0.33; Phoenix, which flagged nothing). It was not lowered, and the rule then failed 0 of 3 where it could be tested (`replication/REPLICATION.md`). Both facts are stated because a pre-registered threshold that is never tested against temptation is not evidence of anything.

Held-out discipline is mechanical. Outcome data was never observed for seven projects — Ozone, Tez, ZooKeeper, Ranger, Oozie, Knox, Sqoop. The rule is written down (`PROJECT_STATE.md` § 3); the ticket-side script requests one Jira field and aborts if it receives another; the matcher-validation script asks git for `%H`, `%s` and `%b` and nothing else, so it *cannot* compute a duration. HBase and Phoenix were dropped from the held-out corpus on 2026-07-30 and quarantined, because their committed artifacts are one subtraction away from per-ticket latency even though no outcome was ever observed for either; their coverage numbers remain usable and are reported.

4 Results

4.1 Corpus eligibility is the binding constraint

Twelve of 38 Apache candidates clear the pre-registered 0.80 commit-side bar, and every one of them is Hadoop-ecosystem under the classification set out below (Table 1, Figure 1). The bar was fixed before any project was cloned and never moved.

No project outside the Hadoop ecosystem clears it. The best of them, James, reaches **74.8%**. The spread runs from ShardingSphere at 0.01% to Ozone at 98.3%, with a median of **63.6%** across all 38 and **44.8%** across the 26 the bar rejected.

The ecosystem label is a hand classification, and we tested whether a measured variable does the same work. It does not. "Hadoop-ecosystem" is not a field in any dataset; one of the authors assigned it. The natural replacement is project generation — free metadata, no judgment, and a mechanism, since projects predating the 2019–20 migration of ASF development to GitHub pull requests had longer under the Jira-citation convention. Two generational variables were tried (`scripts/era_separation.py`):

Neither generational variable separates the corpus as well as the hand label, and the graduation date is **missing precisely where it would be needed**: four of the twelve passing projects — Hive, HBase, ZooKeeper and Ozone — entered the ASF as Hadoop subprojects rather than as incubator podlings and have no graduation date at all. The free metadata is not free for the group that matters.

The drop reasons are measured rather than inferred, because both reference channels are counted per repository. Nine projects are dropped with `github_issue_references_dominate`, meaning the GitHub-issue count exceeds the Jira-key count in that repository — ShardingSphere carries 30,746 GitHub-issue references against 5 Jira keys in 49,111 commits. Four fall in `below_bar_narrowly` (≥70%) and the remaining thirteen in `low_commit_message_hygiene`. Across all 26 the commit-side rate runs **0.0%–78.1%** with a median of **44.8%**; the per-project rows are in the replication package.

4.2 The channel that is published is not the channel that is needed — and the gap is mostly arithmetic

Across the same twelve projects, the **commit-side rate runs 82.6–98.3%** while $\mathrm{TRR}_{\mathrm{live}}$ **runs 12.0–69.0%, with none above 69.0%** (Definition 1 and Definition 2, §3.1; Table 1). The headline case:

> **Apache Hive cites a ticket in 97.0% of its 18,213 commits, and 55.8% of its 29,635 tickets are ever cited by one** — $\mathrm{TRR}_{\mathrm{live}}$**.** Nearly perfect commit-side hygiene, and still nearly half the tracker is invisible to a ticket-anchored design.

Most of that gap is not a discipline gap. It is the ceiling (§3.1.4). Hive has **0.61 commits per ticket**, so at 97.0% commit-side its ticket-side rate cannot exceed **59.6%** whatever anyone does — and it reaches **0.94** of that. The pattern holds across all twelve:

- $\mathrm{ceiling}_{\mathrm{live}}$ **binds for every one of them**, running 13.7% (Kylin) to 81.6% (Ranger) — a 6.0× spread;
- $\mathrm{fill}_{\mathrm{live}}$ **is flat**: median **0.89**, range **0.80** (Ranger) to **0.95** (Drill), a spread of only 1.19×;
- so the 5.8× spread in $\mathrm{TRR}_{\mathrm{live}}$ is 6.0× **ceiling and** 1.19× **fill**.

Every ceiling in this section is $\mathrm{ceiling}_{\mathrm{live}}$**, and the frozen estimator does not agree that the bound binds.** the artifact's 38-project table computes $\mathrm{ceiling}_{\mathrm{frozen}}$ against the frozen Jira snapshot and puts three of the twelve at or above 100% — Ozone **176.0%**, Ranger **130.4%**, Knox **100.0%** — where a bound above 100% does not constrain anything. That is not a disagreement about the projects. It is what a 2026 commit window measured against an older tracker produces: the repository holds more citing commits than the snapshot holds tickets, so the arithmetic ceiling exceeds one and stops being a ceiling. The two estimators bound different denominators, and the note accompanying that measurement says so. Neither figure is withdrawn.

Ranked by fill rather than by rate, the ordering changes almost completely: Drill (43.2%, fill 0.95) sits above Ranger (65.4%, fill 0.80). *Every project in the corpus is reaching most of what its commit supply permits.*

This is a mechanism, and it replaces the reading the divergence invites. The tempting story — "these projects file tickets they never work on" — is not what the numbers show. What they show is that **a tracker accumulates tickets faster than a repository accumulates commits**, and once fewer than one commit exists per ticket the ticket-side rate is capped below the commit-side rate by construction. The two rates were never commensurable, and reporting one as though it licensed the other is the error, not the projects' behaviour.

4.2.1 Kylin: the sharpest number in the corpus, and why it is withdrawn as the example. Earlier drafts led with Kylin, at 83.9% commit-side against 12.0% ticket-side, and **that example is withdrawn**: its pinned commit reaches 968 commits beginning 2022-08-01, **7.5% of the 12,937 on the repository's refs**, against a tracker running since 2014. Its 12.0% is therefore measured over a four-year commit window and its 0.16 commits per ticket is a fact about the branch rather than about Kylin. By fill, the measure that isolates discipline, Kylin sits at **0.87**, the corpus median, and is unremarkable.

4.3 The divergence is a property of the population, not of the survivors

Extending the ticket-side measurement to all 38 probed projects, against a frozen tracker snapshot and with an estimator whose end-to-end error is **1.76pp mean and 11.91pp maximum**, gives **no detectable association** between the two rates: $\rho = -0.010$ over the 33 projects with a usable denominator, 95% CI $[-0.35, +0.34]$, permutation $p = 0.958$. That rules out the strong positive relationship a selection bar implicitly assumes, but at 80% power this study detects only $|\rho| \geq$ **0.47**, so **a null is not established**. On the twelve projects measured exactly and live the same correlation is +0.518, a disagreement in sign we report rather than resolve, and the per-project rows, the estimator's validation and the sensitivity analyses are in the replication package.

4.4 The measure replicates independently across corpora seven years apart

The commit-side rate is not a novel measure, and its agreement with an independently constructed dataset is the reason to trust the probe. Four of the five projects overlapping SEOSS 33 agree within **3.3pp** on corpora seven years apart: Hadoop 97.13% → 97.8%, Hive 96.34% → 97.0%, HBase 90.06% → 92.5%, ZooKeeper 87.12% → 90.4%.

Flink differed by **24.1pp** on full history (41.98% → 66.0%), and the scope explanation for it has now been tested. Re-probing the **earliest 12,419 commits** — SEOSS's own change-set count — gives **5,214 / 12,419 = 41.9841%** against their **41.98%**: a gap of **+0.0041pp**, where full history gives +24.1pp. **Five of five overlapping projects agree once scope is matched.** The remaining 25,800 commits run at 77.62%, and Flink's by-year series shows

variable	coverage	separation
Hadoop-ecosystem label (hand)	38 of 38	accuracy **0.868**, recall 1.000, precision 0.706, Fisher $p = 2.3 \times 10^{-6}$
repository start year (computed from the clones)	38 of 38	AUC **0.611**, $p = 0.279$, best-threshold accuracy 0.711
Apache Incubator graduation date	**24 of 38**	AUC 0.289, $p = 0.098$, best-threshold accuracy 0.625

why: 0.0% in each of 2010, 2011, 2012 and 2013, 19.4% in 2014, 66.0% in 2015, and 70–88% every year since (`paper/FLINK_TRUNCATION.md`).

The exactness of that match is not a warning sign. The quantity is a deterministic count over a fixed, identically bounded commit range — not an estimate, and with no sampling error. Two correct implementations of the same well-specified count should agree exactly, and a disagreement would indicate a specification difference rather than noise; exact agreement is only suspicious between estimators that *have* sampling error. The two numbers also arrive by independent paths, one transcribed from a published table and one scanned fresh from a clone at a pinned sha. **It confirms scope alignment and nothing more** — in particular it does not validate the ticket-side estimator of § 4.3, which is a different measurement with its own error, reported there.

4.5 The standard sampling frame cannot express the criterion

GHS is the standard sampling tool for MSR studies. Its 2021 publication reports **735,669 repositories**; a record returned by the live API in 2026 carries **35 fields**. (The two figures are of different vintages and are kept apart deliberately: the index has certainly grown since 2021, and the 2021 paper describes 25 characteristics rather than 35.) Only `totalIssues` and `openIssues` touch issues at all, and both are GitHub-issue counts. There is no field for issue-tracker type, external tracker usage, issue–commit linkage, traceability, or commit-message convention; the one adjacent feature, filtering by issue label, is described in the paper itself as still under development.

A researcher sampling with GHS therefore gets stars, commits, contributors and license, and discovers post hoc that 26 of 38 candidates are unusable. That is what happened here, and § 5 is the taxonomy of the ways it happens.

5 Six ways a project cannot support an issue-linked study

Every mode below was discovered *after* selecting the project, by probing it. None is expressible in any published sampling frame. GHS indexed 735,669 repositories as published in 2021 and exposes 35 fields per record today, of which only `totalIssues` and `openIssues` touch issues, and both are GitHub-issue counts (§ 2.2). All six were found the same way: by reading commit messages, project by project.

5.1 The distinction that makes this a taxonomy rather than a list

Modes 1, 2 and 3 are disqualifying rather than measurable. The project is not a weak candidate; it is not a candidate. Scoring kata-containers as "0% traceable" would place it on the same axis as a project that tries and fails, and would put a number where a category belongs.

Modes 4, 5 and 6 are silent. They produce a plausible-looking low number rather than an error. A single-key probe reads Hadoop as 26.2% and Evergreen as 71.7%, and **nothing in either result signals that the key set is wrong.** Four of the 27 projects in `paper/intersection.json` needed multi-key matching — DATALAB (`DLAB+EPMCDLAB`), TRINIDAD (`TRINIDAD+ADFFACES`), DAOS (`DAOS+CART`), EVG (`EVG+DEVPROD`).

This is the practical consequence, and it is the section's point: **project eligibility for issue-linked research cannot be established from metadata.** It requires reading commit messages from the project itself, per project, before any outcome is measured.

5.2 What a sampling frame would need

None of the six is derivable from repository metadata. Expressing them requires, per candidate: the dominant reference channel (Jira keys against GitHub issues, counted rather than assumed), the tracker's project-key set, the key set actually appearing in commit messages, and the rate computed separately over recent history to expose mode 4. All four are cheap. None is in GHS.

One qualification, because the unqualified claim overstates the gap. We say above that the key set cannot be recovered from the tracker, and that stands: a key cited only in git — `OPTIQ`, `EPMCDLAB`, `DEVPROD` — is not in the tracker to be enumerated. But a substantial literature *does* recover missing issue–commit links without relying on the commit message, beginning with ReLink [11] (Wu et al., ESEC/FSE'11) and continuing since, by learning from time proximity, author identity and textual similarity between report and change (§ 2.3). Those methods recover **links**, not **key sets**, and they presuppose a project already known to be a candidate — which is the step this taxonomy is about. Modes 4–6 make a project's *measured rate* wrong; link recovery can raise the true rate afterwards. A sampling frame that exposed the four fields above would tell a researcher which projects are worth pointing a link-recovery tool at, which is a different and prior question.

6 Threats to validity

Several of the items below are things this study got wrong and corrected. **They are stated as the paper's credibility, not as its embarrassments** — a methods paper about record quality that concealed its own record would be self-refuting.

6.1 Construct validity

The 38-project extension uses a different denominator source, and the validation figure the earlier draft quoted was for a different approximation. § 4.3 measures the ticket realisation rate for all 38 projects against the frozen public Jira corpus rather than a second live fetch, aligning numerator and denominator in issue-number space (§ 3.1.3). **Two approximations are stacked**: number-capping, and substituting a frozen snapshot for the live tracker. The 0.45pp / 2.14pp figure earlier drafts quoted validates only the first. The end-to-end error of the combination the artifact's

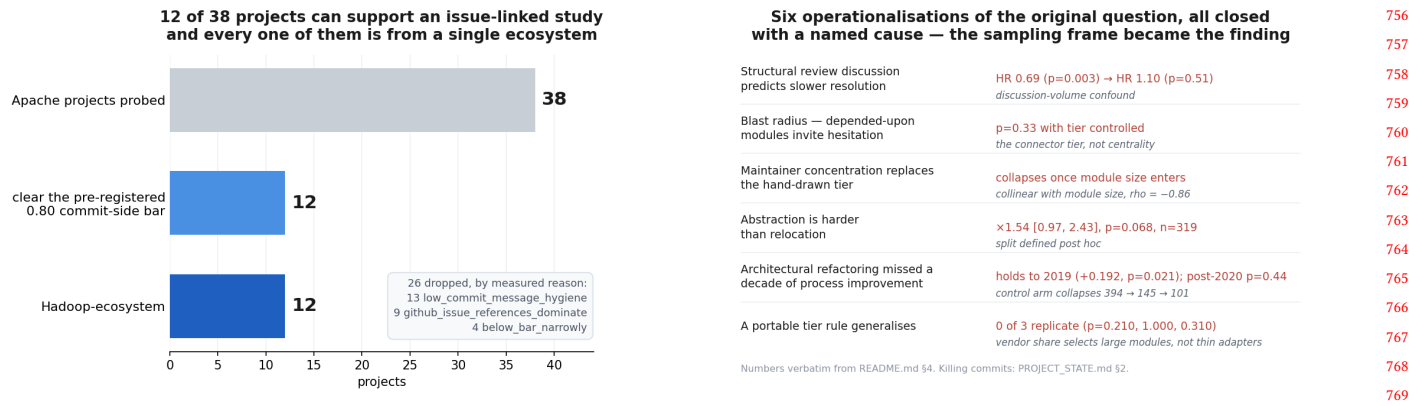

Figure 1: Corpus attrition: 38 probed projects to 12 eligible to a single ecosystem, with the six operationalisations that were tested and closed.

#	mode	worked example	evidence
1	**GitHub Issues displaced Jira**	ShardingSphere: **5** Jira citations in 49,111 commits, against 30,746 GitHub-issue references	`paper/traceability_probe.json`
2	**No source repository exists**	RedHat RHBRMS: 86.00% estimate coverage on 2,400 issues — a product/documentation tracker with no code	`estimates_by_org.json`; repo resolution failed
3	**The tracker is downstream of the upstream repo**	kata-containers: **zero** `KATA-` keys in 19,807 commits; the Red Hat Jira tracks a product built from an upstream nobody asks to cite	`paper/intersection.json` (excluded, not scored)
4	**Convention changed mid-history**	spring-batch: `BATCH-` in **45.6%** of 7,035 commits overall, in **0 of the most recent 1,000** (back to 2021-06), and at exactly **0% in every year since 2020** — a single rate averages two regimes	full scan, `scripts/springbatch_recency.py`
5	**Monorepo needs multi-key matching**	Hadoop: **26.2%** single-key → **92.3%** four-key → **97.8%** seven-key, same 28,290 commits	`scripts/citation_rate.py`
6	**The cited key has no project record in the tracker**	Evergreen: commits cite `DEVPROD` (2,785) but the tracker holds only `EVG`. DataLab: commits cite `EPMCDLAB` (3,900) and `DLAB` (3,547); the tracker holds only `DATALAB`	`paper/intersection.json`

38-project table actually uses is **1.76pp mean / 11.91pp max**, the worst case being Kylin; excluding Kylin it is 0.84pp mean and **2.93pp max, with 11 of 12 inside 3pp**. **Quote the end-to-end figure.**

And the estimator is applied entirely outside its validated range. The twelve validation projects span commit-side **82.6–98.3%**; the twenty-one that carry the §4.3 association span **10.5–78.1%**. The ranges are **disjoint — 0 of 21** applied projects fall inside the validated one. The error mechanism is tracker-numbering density rather than commit hygiene, and within the twelve the error does not track the commit-side rate ($\rho = -0.16$, $n = 12$), so there is a reason to expect the extrapolation to hold — but it is a reason, not a validation, and **any claim whose margin is smaller than 11.91pp is not safe on this estimator.** The claims that are: the ceiling identity (exact arithmetic, not estimated); the eligibility result (commit-side only); the taxonomy. The claims that are not: any per-project ticket-side comparison in the artifact's 38-project table closer than ~12pp, and the passing-versus-dropped median gap of 0.6pp, which is far inside the noise and is reported as such.

Four further things can break the estimator, and all four occur in this corpus:

(1) **Trackers with gaps.** Number-capping assumes a tracker holding N issues holds approximately the first N numbers. Issues moved or deleted break that, and the estimator then undercounts. This is the largest single validation error in the set and the direction is always downward.

(2) **Repositories whose history does not span the snapshot era.** Where a repository has been truncated or re-initialised, almost none of its cited keys fall inside the frozen range and the estimated rate collapses towards zero. That is a *true* statement about what is recoverable from the repository as it now stands, and a badly misleading one if read as a statement about whether the project's tickets were ever worked. Affected projects are flagged in the artifact's 38-project table with the share of cited keys that postdate the snapshot — **99.72% for Kylin (709 of 711), not 100%**; an earlier draft printed 100%, which contradicts the non-zero numerator that produces Kylin's 0.04%. **This defect is not confined to the frozen arm, and that is the more serious finding.** Kylin's pinned commit reaches **968 commits beginning 2022-08-01**, **7.5% of the 12,937 on the repository's refs**, against a tracker of 5,931 issues and a repository that begins 2014-05-13. Its *live* 12.0% is therefore also measured over a four-year window against a twelve-year tracker. Earlier drafts made that 12.0% the paper's flagship example while dismissing the frozen 0.04% as an artefact; **both are the same artefact**, and the example has been moved to Hive (§4.2.1). Three other projects have a pinned

branch starting after their repository — Jena (49.0% of refs), Karaf (45.4%), James (89.4%, by eight days) — and none approaches Kylin's severity (`scripts/revision_metrics.py`).

(3) **Denominators too small to carry a rate.** Several trackers hold only a handful of issues in the frozen corpus. Projects with fewer than 500 are excluded from the association statistic and shown in the table with the exclusion marked.

(4) **Keys with no tracker record at all.** Some probed keys have no project record in the frozen corpus. These are taxonomy modes 1 and 6, not low rates, and they are **excluded rather than scored as 0%** — scoring them would put a number where a category belongs, which is the error §5.1 exists to prevent.

The truncation caveat on Table 1 is not withdrawn. The 12.0–69.0% range from the live measurement remains within-passing variation, computed on the twelve that had already cleared the bar. §4.3 does not extend that measurement; it is a different estimator against a different snapshot, and it is reported alongside rather than merged into it.

6.2 Single-rater limits, and the one measure that escapes them

Inter-rater agreement cannot be computed by one person, so no measure requiring manual coding carries a claim in this paper. Two instruments are affected and both are reported **as findings about the instrument**, never as instruments the paper's claims rest on:

- the codebook's keyword rule for structural review discussion — **≈25% precise** (5 of 20 flagged comments genuinely structural), ≈5% miss (1 of 20 unflagged), on a 40-comment single-rater pass, with no κ;

Its role is to establish that keyword-based detection over-counts roughly fourfold, which is a validity result about a method other studies use.

The exception is argued rather than asserted. The 200-commit matcher check is a mechanical determination against a written rule with categories fixed before labelling, not a coded judgment where the construct lives in the rater's head. It is made **auditable rather than agreed**: `paper/matcher_sample.json` carries all 200 messages and `paper/matcher_labels.json` every label, so any reader can re-check the sample without redrawing it.

The architectural-episode gold set has no second rater and is therefore not used to support any claim here.

One coincidence in the rater pilot is disclosed because it looks like tuning and we cannot prove it is not. `paper/LLM_RATER_PILOT.md` reports that six comments change label under the opposite ordering of two codebook rules, and **five of the six fall in the flagged bucket** — the margin moves 3/12/5 to 3/7/10, a swing of exactly five. The κ ceiling is computed on that bucket, so five is the count that drives it, and five moves the attainable ceiling from 0.491 to **0.840**. *(Corrected 2026-08-05: this paragraph previously credited the 0.840 to the six re-labelled comments. Six is the total across both buckets; the ceiling is indexed by flagged-bucket moves alone. No figure changes.)*

Five is also exactly the count that maximises the ceiling over every possible count: four gives 0.765, six gives 0.837, seven gives 0.833. The six were selected by a stated semantic criterion, they are individually defensible, and the finding is robust at 0.833–0.840 across plus or minus two comments — but the criterion was applied by the same party that reported the resulting number, and it landed on the global maximum. Stated here rather than left for a reader to find (`audit/JUDGMENTS.md` §3).

6.3 External validity

One mined project, one ecosystem. All 12 eligible projects are Hadoop-ecosystem, and depth beyond Hadoop was not reachable. The corpus bound is therefore a finding and a limit at once: results about Apache/Jira/Java traceability generalise to Apache/Jira/Java.

At 12 projects the design is not powered, whatever the corpus size. Effective n is capped at P/ICC by between-project correlation — a ceiling of 600 at ICC 0.02 against the 769 the effect needs. The direction of the bias is bad: these twelve share contributors, committers, review norms and in several cases build and CI infrastructure, so this sample's ICC is higher than a random sample's. **The ICC itself is unmeasured, and was deliberately not estimated from the four available clusters**, because between-cluster variance is not estimable at $n = 4$ and the estimate must come from the pooled study as a first stage (`replication/CORPUS_FEASIBILITY.md`).

7 Discussion

7.1 For researchers designing an issue-anchored study

Project eligibility cannot be established from metadata. Every one of the six failure modes in §5 was found by reading commit messages from the project itself, after the project had been selected. Three of them are silent: they yield a plausible low number rather than an error, and nothing in the result signals that the key set is wrong. A researcher who samples on stars, contributors and language, then computes a linkage rate, will get a number for every candidate and will not be told which of those numbers mean anything.

Measure the side of the rate your design actually samples on, and report commits per ticket beside it. A commit-side rate answers "if I start from a commit, can I find its ticket?". A design that starts from *tickets* needs the ticket realisation rate, and the first does not imply the second — Hive is 97.0% one way and 55.8% the other ($\mathrm{TRR}_{\mathrm{live}}$). But the more useful advice is the ceiling (§3.1.4): **below one commit per ticket the two rates are not commensurable at all**, and every project in our eligible corpus is below it. Report `commits / tickets`; it costs nothing, it bounds the ticket-side rate before any measurement, and no published linkage rate carries it.

What our own extension does and does not license. Across the whole probe we find no detectable association between the two rates ($\rho = -0.010$, $n = 33$, 95% CI $[-0.35, +0.34]$). That rules out a strong positive relationship — the assumption a selection bar encodes — but at 80% power this study detects only $|\rho| \geq 0.47$, so it does not establish a null. On the twelve projects measured exactly

the same correlation is +0.518, and we report the disagreement rather than resolving it (`paper/DIRECTION_TENSION.md`).

Report the bar, the attrition, and the rejected candidates. The dropped projects are the informative part. That 26 of 38 Apache candidates are unusable, and that the 12 survivors are one ecosystem, is a fact about the population that a paper reporting only its final corpus cannot convey — and it is a fact each such paper independently rediscovers and does not write down.

7.2 For dataset and sampling-frame builders

The fields that decide usability are absent from the standard frame. GHS indexed 735,669 repositories as published in 2021 and exposes 35 fields per record today; only `totalIssues` and `openIssues` touch issues and both are GitHub-issue counts. There is no tracker type, no external tracker, no issue–commit linkage, no commit-message convention.

Four fields would close most of the gap, and all four are cheap to compute from a shallow clone and a tracker listing:

(1) **the dominant reference channel** — Jira-key citations against GitHub-issue references, *counted*, not assumed;
(2) **the tracker's project-key set**, which catches concurrent siblings (mode 5);
(3) **the key set actually appearing in commit messages**, which is the only thing that catches superseded records (mode 6) — those keys exist in git and nowhere else;
(4) **the rate recomputed over recent history**, which exposes a convention that changed mid-history (mode 4).

This study spent 38 clones and 4.3 GB to keep 12. That cost is paid again by every group that attempts the same kind of corpus, and none of it is recoverable from published metadata.

7.3 What follows from the corpus limit, without softening it

A corpus bounded to one ecosystem cannot support the between-ecosystem comparison the design wanted, and there are three ways out, each with a cost: **lower the bar and model the measurement error**, which buys nine more projects at the price of a biased sample of tickets within each; **change the outcome so it needs no tickets**, which dissolves the problem and the creation-to-first-commit clock with it; or **accept ecosystem-bounded scope and say so**. The third is the honest one and is what we do.

7.4 What this paper does not claim

It does not claim that Jira-citing projects are better engineered, or that the projects on GitHub Issues are less traceable in any sense that matters to their own maintainers. The convention this research needs is not a quality signal; it is a research affordance, and its decline is a cost to researchers rather than to the projects.

It does not propose the successor design that the corpus limits point towards. That is a separate registered report with its own feasibility gates, and its novelty margin against an active architectural-debt time-to-fix literature is recorded in the replication package as **not assessed**.

And it does not offer a fix. The six modes are diagnosable, not repairable: a project whose tracker is downstream of its repository will not start citing keys because a researcher would like it to.

8 Acknowledgements and disclosures

8.1 Data availability

Every artifact this paper rests on is archived and citable: the 38 pinned repository shas, the frozen tracker caches, the per-project measurements, the full 38-row eligibility table, the three-channel visibility table, every script that produced a number here, and the audit report against which the numbers were re-derived. **Replication package: ANONYMISED-ARTIFACT-DOI.** Nothing needed to reproduce a figure in this paper lives only in a version-control host.

8.2 Data and tools

The Apache Software Foundation publishes the trackers and repositories the corpus is drawn from. The Public Jira Dataset (Zenodo 15719919, CC BY 4.0) supplies every frozen ticket denominator, SEOSS 33 provides the independent measurement the probe is validated against, and RefactoringMiner is the detector.

8.3 Use of AI tools

Large language model assistants (Anthropic's Claude) were used in this work: to write and run the analysis code, every script of which is committed and re-runnable; to draft prose from the author's outlines, findings and decisions; to check citations against source texts; and to conduct an adversarial audit that re-derived the load-bearing results with separately written code and reported the errors it found, unedited, in the replication package. The research questions, the study design, the pre-registered bar and its commitment before any project was cloned, the six retractions, and the judgment of what the results mean are the author's. **The author takes full responsibility for all content, including any error a tool introduced and the author did not catch.**

8.4 A separate matter: the LLM used as a rater

Section 6.2 reports a pilot in which a language model applied this study's codebook as a second rater. **That is an object of study here, not a method the paper's claims rest on**, and it is reported with its contamination disclosed and its conclusions bounded. No claim in this paper depends on it.

8.5 Funding and competing interests

No funding was received. The authors declare no competing interests.

Tables

12 of 38 projects clear the pre-registered 0.80 commit-side bar. All 12 are Hadoop-ecosystem.

Eligible (12)

† held-out corpus: coverage and existence counts only, no outcome ever observed (`PROJECT_STATE.md` §3). ‡ quarantined 2026-07-30,

Table 1: Corpus eligibility across the 38 probed projects, both traceability channels, with commits per ticket, the arithmetic ceiling of Section 3.1.4 and the share of it reached. All quantities are the live measurement: $\mathrm{TRR}_{\mathrm{live}}$, $\mathrm{ceiling}_{\mathrm{live}}$, $\mathrm{fill}_{\mathrm{live}}$ (Section 3.1.3).

#	project	multi-key	commits/ticket	ceiling	ticket-side	TRR/ceiling
1	ozone †	**98.3%**	0.69	67.7%	**63.9%**	0.94
2	tez †	**97.4%**	0.66	64.4%	**58.7%**	0.91
3	hive	**97.0%**	0.61	59.6%	**55.8%**	0.94
4	hbase ‡	**92.5%**	0.71	65.3%	**55.6%**	0.85
5	phoenix ‡	**92.0%**	0.54	49.5%	**41.3%**	0.84
6	zookeeper †	**90.4%**	0.56	50.4%	**47.0%**	0.93
7	ranger †	**86.3%**	0.95	81.6%	**65.4%**	0.80
8	oozie †	**85.2%**	0.65	55.1%	**49.7%**	0.90
9	knox †	**84.3%**	0.94	79.4%	**69.0%**	0.87
10	drill	**84.2%**	0.54	45.4%	**43.2%**	0.95
11	kylin	**83.9%**	0.16	13.7%	**12.0%**	0.87
12	sqoop †	**82.6%**	0.31	25.4%	**21.2%**	0.84

dropped from the held-out corpus; coverage retained, outcome inputs sequestered in `spent/` (`spent/README.md`).

References

[1] Adrian Bachmann, Christian Bird, Foyzur Rahman, Premkumar Devanbu, and Abraham Bernstein. 2010. The missing links: bugs and bug-fix commits. In *Proceedings of the 18th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 97–106. doi:10.1145/1882291.1882308

[2] Christian Bird, Adrian Bachmann, Eirik Aune, John Duffy, Abraham Bernstein, Vladimir Filkov, and Premkumar Devanbu. 2009. Fair and balanced? Bias in bug-fix datasets. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*. 121–130. doi:10.1145/1595696.1595716

[3] Christian Bird, Adrian Bachmann, Foyzur Rahman, and Abraham Bernstein. 2010. LINKSTER: enabling efficient manual inspection and annotation of mined data. In *Proceedings of the 18th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 369–370. doi:10.1145/1882291.1882352

[4] Ozren Dabic, Emad Aghajani, and Gabriele Bavota. 2021. Sampling Projects in GitHub for MSR Studies. In *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*. 560–564. arXiv:2103.04682. doi:10.1109/MSR52588.2021.00074

[5] Kim Herzig, Sascha Just, and Andreas Zeller. 2013. It's not a bug, it's a feature: how misclassification impacts bug prediction. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 392–401. doi:10.1109/ICSE.2013.6606585

[6] Lloyd Montgomery, Clara Lüders, and Walid Maalej. 2025. The Public Jira Dataset. Zenodo. Licensed CC BY 4.0. doi:10.5281/zenodo.15719919

[7] Thanh H. D. Nguyen, Bram Adams, and Ahmed E. Hassan. 2010. A Case Study of Bias in Bug-Fix Datasets. In *Proceedings of the 17th Working Conference on Reverse Engineering (WCRE)*. 259–268. doi:10.1109/WCRE.2010.37

[8] Michael Rath and Patrick Mäder. 2019. The SEOSS 33 dataset — Requirements, bug reports, code history, and trace links for entire projects. *Data in Brief* 25 (2019), 104005. doi:10.1016/j.dib.2019.104005

[9] Michael Rath, Jacob Rendall, Jin L. C. Guo, Jane Cleland-Huang, and Patrick Mäder. 2018. Traceability in the wild: automatically augmenting incomplete trace links. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. 834–845. arXiv:1804.02433. doi:10.1145/3180155.3180207

[10] Renan Gomes Vieira, Antônio da Silva, Lincoln S. Rocha, and João Paulo Pordeus Gomes. 2019. From Reports to Bug-Fix Commits: A 10 Years Dataset of Bug-Fixing Activity from 55 Apache's Open Source Projects. In *Proceedings of the 15th International Conference on Predictive Models and Data Analytics in Software Engineering (PROMISE)*. 80–89. doi:10.1145/3345629.3345639

[11] Rongxin Wu, Hongyu Zhang, Sunghun Kim, and Shing-Chi Cheung. 2011. ReLink: recovering links between bugs and changes. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*. 15–25. doi:10.1145/2025113.2025120